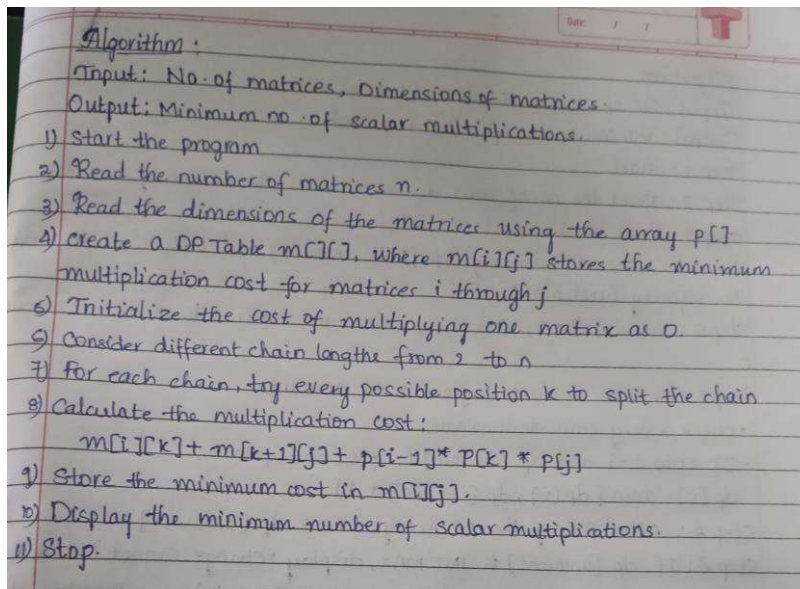


Practical –6

Aim: Implementation of chain matrix multiplication using dynamic programming.

Algorithm:



Program:

```
#include using namespace std;

int main () {

    int n, p [20], c [20][20]; int i, j, k, d, q, min;

    cout << "Enter number of matrices: ";
    cin >> n;

    cout << "Enter " << n + 1 << " dimensions: ";
    for (i = 0; i <= n; i++)
    {
        cin >> p[i];
    }

    for (i = 1; i <= n; i++)
    {
```

Name:V.Bhavya sri
Enrollment no : 92460118227

```
c[i][i] = 0;
}

for (d = 1; d < n; d++)
{
    for (i = 1; i <= n - d; i++)
    {
        j = i + d;
        min = 32767;

        for (k = i; k <= j - 1; k++)
        {
            q = c[i][k] + c[k + 1][j]
                + p[i - 1] * p[k] * p[j];

            if (q < min)
            {
                min = q;
            }
        }

        c[i][j] = min;
    }
}

cout << "Minimum number of scalar multiplications = "
    << c[1][n] << endl;

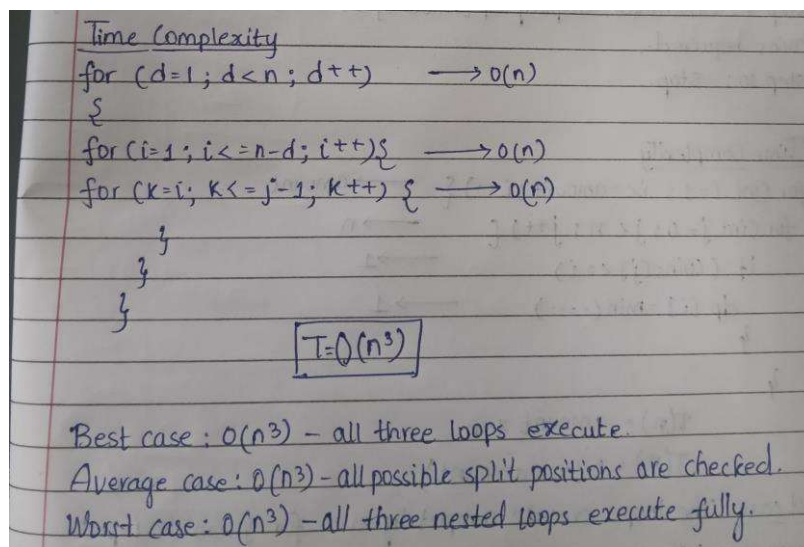
return 0;

}
```

Output:

```
Enter number of matrices: 3
Enter 4 dimensions: 4
8
7
10
Minimum number of scalar multiplications = 504
```

Time complexity:



Conclusion:

The Matrix Chain Multiplication problem was successfully implemented using Dynamic Programming. The method finds the optimal order of matrix multiplication and minimizes the number of scalar multiplications, with a time complexity of $O(n^3)$ and space complexity of $O(n^2)$.